OMNIX QUANTUM LTD

PROVISIONAL PATENT APPLICATION

OMNIX-PAT-2026-015 | 35 U.S.C. § 111(b) | Micro Entity | April 19, 2026

STRUCTURAL ADMISSIBILITY ENGINE FOR AUTOMATED DECISION GOVERNANCE SYSTEMS WITH PRE-PIPELINE SCHEMA VALIDATION, ENUMERATED CONSTRAINT ENCODING, AND ZERO-BYPASS BOUNDARY ENFORCEMENT

Inventor:	Harold Alberto Nunes Rodelo	**Docket:**	OMNIX-PAT-2026-015
Applicant:	OMNIX Quantum Ltd	**Filing Basis:**	35 U.S.C. § 111(b)
Jurisdiction:	United Kingdom	**Entity Status:**	Micro Entity
Date Filed:	April 19, 2026	**Related Apps:**	OMNIX-PAT-2026-001, -011, -014

“OMNIX does not only block invalid decisions — it decides which decisions can exist.”

FIELD OF THE INVENTION

The present invention relates to automated decision governance systems, and more particularly to a pre-pipeline structural admissibility layer that enforces decision admissibility at the point of input construction rather than at runtime evaluation — making structurally inadmissible decision requests unrepresentable as valid system objects. The Structural Admissibility Engine (SAE) constitutes a new architectural stratum — **Layer 0** — that precedes and gates all downstream governance processing, providing a zero-bypass guarantee that no inadmissible request can enter the evaluation pipeline regardless of the operational state of any downstream component.

The invention is domain-agnostic and applies without modification to automated governance pipelines in financial trading, insurance underwriting, medical AI, real estate, energy, and autonomous agent systems. The SAE integrates with the four-layer governance architecture documented in related applications, constituting the constitutive boundary layer that determines what requests may exist in the system before any evaluative processing occurs.

BACKGROUND OF THE INVENTION

I. THE FUNDAMENTAL INADEQUACY OF RUNTIME INTERCEPTION

Contemporary automated decision governance systems — including multi-checkpoint sequential pipelines, rule-based policy engines, and AI-driven compliance systems — share a common structural assumption: that governance is enforced by intercepting and blocking invalid decisions *after* those decisions have been formulated and submitted for evaluation. This runtime interception model has five irremediable architectural deficiencies:

1.1 Bypass Through Component Failure. In a runtime interception model, every blocking component must be operational and correctly implemented for the governance guarantee to hold. If a governance checkpoint fails silently, is disabled for maintenance, encounters an unhandled exception, or is bypassed by a

misconfigured pipeline, the invalid decision it was designed to block may proceed to execution. The governance guarantee is only as strong as the weakest component in the chain.

1.2 Latent Invalid State Generation. Because the invalid decision is formulated as a system object before any checkpoint evaluates it, the system must allocate computational resources to represent, transmit, and process a request that is definitionally inadmissible. This generates latent invalid state throughout the system — in memory, in logs, in audit trails — even when the request is ultimately blocked.

1.3 Governance Dependency on Execution Order. Runtime interception pipelines depend on correct execution ordering to guarantee governance. A checkpoint evaluating jurisdictional compliance must execute before one generating an execution commitment. Any inversion — through code defect, configuration error, or concurrent execution — may permit an inadmissible decision to receive an execution commitment before the blocking checkpoint evaluates it.

1.4 Absence of a Constitutive Boundary. In existing systems, the boundary between admissible and inadmissible requests is not constitutive — it does not determine what can exist in the system. It is merely evaluative — it determines what happens to things that already exist. No architectural mechanism prevents the construction and submission of a request for an operation that is categorically prohibited by jurisdiction, asset class, operation type, regulatory classification, or ethical constraint.

1.5 No Constraint Provenance in Rejection. When a runtime interception system blocks a request, the rejection is typically expressed as an error code or a binary BLOCKED decision. The system does not identify the specific structural constraint violated, the regulatory source from which it derives, or the combination of input fields that jointly produced the inadmissibility. This impedes regulatory audit, client communication, and system debugging.

II. ILLUSTRATIVE FAILURES OF RUNTIME INTERCEPTION

The severity of the runtime interception model's deficiencies is illustrated by documented incidents in automated systems where governance failures occurred despite the existence of checkpoint components:

Knight Capital Group (August 1, 2012). A loss of approximately $440 million USD in 45 minutes resulted from erroneous activation of a legacy software module not properly decommissioned. No architectural mechanism detected the divergence between intended and actual system behavior at the point of execution. A constitutive boundary would have prevented construction of the invalid execution context.

Flash Crash (May 6, 2010). Automated trading algorithms responded to market conditions in mutually reinforcing patterns without any governance layer capable of detecting the systemic coherence failure. The invalid trading context existed and was processed before governance could respond.

Zillow Offers Algorithm (2021). The system continued making commitments at prices contradicting available market signals, accumulating a $304 million write-down. A structural admissibility layer encoding market coherence constraints would have prevented invalid purchase requests from being constructed.

III. PRIOR ART AND ITS LIMITATIONS

Runtime Checkpoint Pipelines (OMNIX-PAT-2026-001). Sequential checkpoints evaluate decisions against specific criteria and produce pass/block determinations. These systems operate entirely within the runtime interception model. They are powerful but do not prevent invalid requests from being formulated.

Web API Schema Validation (JSON Schema, OpenAPI). Web API schema validation systems (JSON Schema, OpenAPI Specification, Pydantic) validate that input data conforms to a *structural* specification: that a field named "asset" contains a string, that a numeric field falls within a range, that required fields are present. **This is categorically distinct from decision admissibility.** JSON Schema answers: "Is this input

well-formed data?" The SAE answers: "Is this decision admissible under the applicable regulatory, jurisdictional, and ethical framework?" JSON Schema cannot determine that XMR (Monero) is PROHIBITED in UAE jurisdiction under VARA regulations, or that a LEVERAGED operation is PROHIBITED in UK for retail clients under FCA rules, or that a specific asset is HARAM under Sharia compliance criteria. These determinations require regulatory constraint encoding, not data structure validation. The present invention performs regulatory admissibility determination, not schema validation.

Type Systems in Programming Languages. The principle "make illegal states unrepresentable" is known in functional programming and type-theoretic literature (Minsky, 2010; Wadler, 1989). It has been applied to data modeling in general software applications. **Critically, this principle has never been applied as a dedicated architectural layer within automated decision governance systems.** No prior art encodes regulatory constraints (jurisdiction-asset, jurisdiction-operation, Sharia, ESG, sanctions) at the type or schema level within a governance pipeline, nor provides the zero-bypass architectural guarantee, nor produces structured rejection with regulatory constraint provenance. The present invention is the first application of structural admissibility as a constitutive governance boundary for automated decision systems operating under regulatory frameworks.

Access Control Frameworks (OAuth, RBAC, ABAC). Authorization frameworks control whether an entity is permitted to submit a request. They operate at the identity and permission level, not at the decision content level. An authorized user may still submit a structurally inadmissible decision request.

The critical distinction: JSON Schema validates whether data is well-formed. Type systems prevent invalid data states in general software. Access control determines who may submit a request. **None of these determine whether a proposed decision is admissible under a regulatory, jurisdictional, or ethical framework — and none provide a zero-bypass architectural guarantee that inadmissible decisions cannot be constructed as system objects in a governance pipeline.** The present invention is the first system to apply structural admissibility as a constitutive pre-pipeline governance boundary with regulatory constraint encoding, zero-bypass enforcement, and structured rejection with constraint provenance for automated decision systems operating under regulatory frameworks across multiple domains.

SUMMARY OF THE INVENTION

The present invention provides a Structural Admissibility Engine comprising five components that together enforce the principle that inadmissible decision requests cannot be represented as valid system objects:

Component A — Structural Constraint Schema (SCS): A declarative, machine-readable specification of all constraints determining whether a proposed decision request is structurally admissible. Organized into four classes: (i) Jurisdiction-Asset Constraints; (ii) Jurisdiction-Operation Constraints; (iii) Ethical Compliance Constraints (Sharia, ESG, Sanctions); and (iv) Client-Specific Constraints.

Component B — Structural Admissibility Validator (SAV): A pre-construction validator that evaluates a proposed decision request against the SCS before constructing the EvaluationRequest object. If any constraint in any class is violated, the SAV does not construct the object. The invalid request is rejected at the boundary — it never becomes a valid system object.

Component C — Zero-Bypass Boundary Enforcement (ZBE): An architectural guarantee that the SAV is the sole path through which a proposed decision request may be converted into an EvaluationRequest object. EvaluationRequest construction is private to the SAV. No external code path can construct an EvaluationRequest without passing through full constraint evaluation.

Component D — Structured Rejection with Constraint Provenance (SRCP): A machine-readable rejection record identifying: the specific constraint violated; its constraint class; its regulatory or policy source; the specific input fields jointly responsible; and a resolution guidance field for actionable client communication.

Component E — Composable Cross-Domain Constraint Architecture (CCCA): A constraint composition mechanism allowing constraints from multiple domains and sources to be combined into a single SAV evaluation without runtime branching logic. New constraint classes are added to the Constraint Registry without modifying the SAV evaluation logic.

Figure 1 — Four-Layer Governance Architecture (OMNIX-PAT-2026-015)

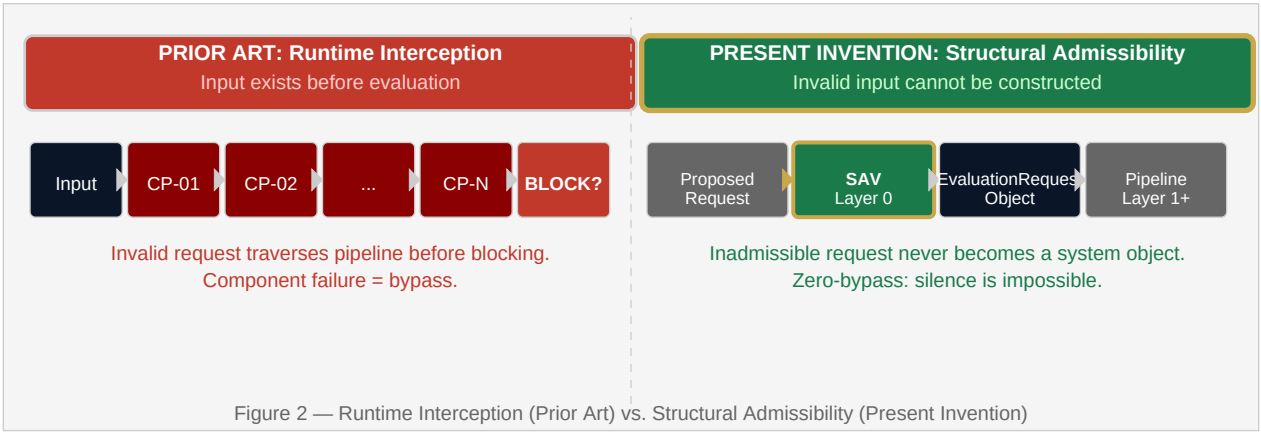

Figure 2 — Runtime Interception (Prior Art) vs. Structural Admissibility (Present Invention)

DETAILED DESCRIPTION OF THE INVENTION

I. LAYER 0 IN THE FOUR-LAYER GOVERNANCE ARCHITECTURE

Layer	Name	Role	Related Patent
0	Structural Admissibility Engine (present invention)	Constitutive: determines what requests can exist in the system	OMNIX-PAT-2026-015
1	OMNIX Runtime Pipeline	Evaluative: CP-0 through CP-11 + TIE sequential veto-authority checkpoints	OMNIX-PAT-2026-001
2	Trajectory Invariant Engine	Temporal: retrospective + prospective admissibility assessment	OMNIX-PAT-2026-014
3	Evidence & Receipt Layer	Cryptographic: PQC-signed immutable audit receipt (Dilithium-3)	OMNIX-PAT-2026-001

Key architectural distinction: Layers 1, 2, and 3 evaluate requests that *exist*. Layer 0 determines what *can exist*.

II. THE STRUCTURAL CONSTRAINT SCHEMA (SCS)

The SCS is a declarative, in-memory specification organized in a hierarchical Constraint Registry. For each constraint class, the SCS encodes admissibility as a function of the relevant input fields:

Class A — Jurisdiction-Asset Constraints

For each regulatory jurisdiction J and each asset class A, the SCS specifies JA_CONSTRAINT(J, A) $\in$ {PERMITTED, PROHIBITED, CONDITIONAL}. Jurisdictions include UAE (VARA framework), EU (MiCA), US (FinCEN/SEC/CFTC), UK (FCA), GCC, SG (MAS), and GLOBAL. Privacy coins (XMR, ZEC, DASH, GRIN, BEAM, FIRO) are PROHIBITED in UAE, EU, US, UK, GCC, and SG jurisdictions under applicable AML regulations. Assets on active OFAC, EU, or UN sanctions lists are PROHIBITED in all jurisdictions regardless of other constraints.

Class B — Jurisdiction-Operation Constraints

For each jurisdiction J and operation type O (SPOT, LEVERAGED, DERIVATIVES, SHORT, STAKING, LENDING), JO_CONSTRAINT(J, O) $\in$ {PERMITTED, PROHIBITED, CONDITIONAL}. Examples: UAE LEVERAGED = PROHIBITED (VARA); US DERIVATIVES = CONDITIONAL (requires CFTC authorization); EU DERIVATIVES = PERMITTED (MiCA). CONDITIONAL determinations require all associated conditions to be satisfied to resolve to PERMITTED.

Class C — Ethical Compliance Constraints

Sharia compliance constraints encode a determination S(A, O) $\in$ {HALAL, HARAM, MASHBOOH} for each asset-operation combination. HARAM determinations are PROHIBITED for clients with Sharia compliance requirements. ESG screening constraints are configurable per client. Sanctions constraints (OFAC, EU, UN) are PROHIBITED for all clients. See also OMNIX-PAT-2026-003 (Ethical and Quantum-Secure Execution Framework).

Class D — Client-Specific Constraints

Per-client constraints negotiated at onboarding and stored in a client constraint record: CLIENT_CONSTRAINT(client_id, field, value) $\rightarrow$ {PERMITTED, PROHIBITED}. Client constraints may further restrict but may not expand the default structural admissibility. A client may prohibit additional assets beyond the jurisdictional default, but cannot permit assets that are categorically prohibited by jurisdiction or ethical constraint.

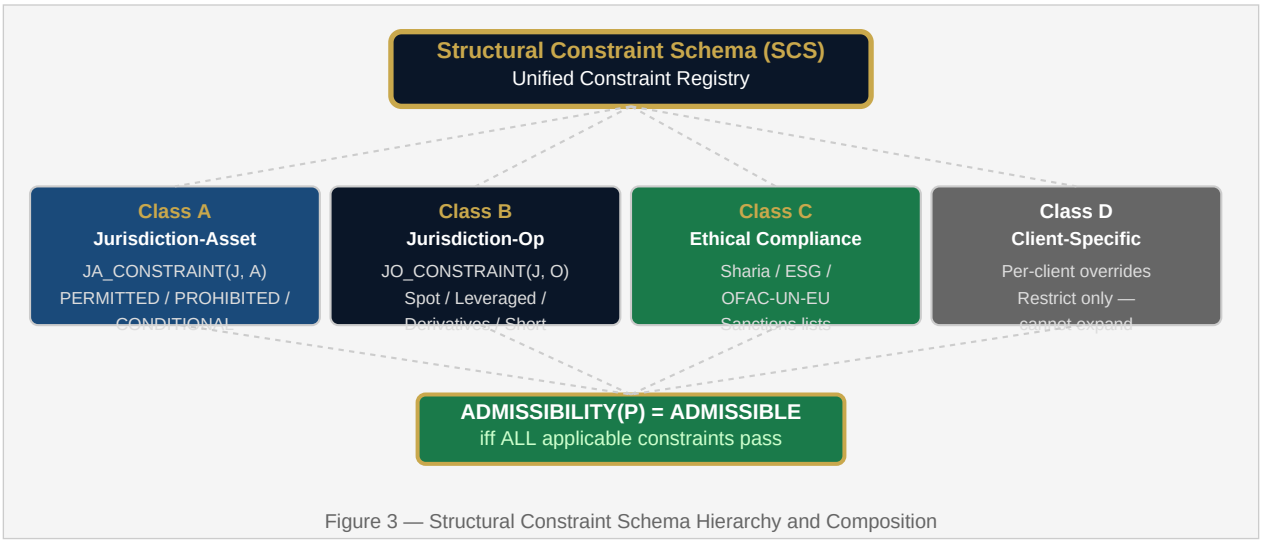

Figure 3 — Structural Constraint Schema Hierarchy and Composition

III. THE STRUCTURAL ADMISSIBILITY VALIDATOR (SAV)

When external code submits a proposed decision request P, the SAV performs full constraint evaluation before constructing any EvaluationRequest object:

```
ADMISSIBILITY(P) = ■ { C_i(P) : C_i ∈ SCS applicable to P }
```

If any applicable constraint determines P to be PROHIBITED, ADMISSIBILITY(P) = INADMISSIBLE. The SAV does not construct an EvaluationRequest object and returns a Structured Rejection Record. If ADMISSIBILITY(P) = ADMISSIBLE, the SAV constructs and returns an EvaluationRequest object — the only valid input to Layer 1 of the governance pipeline.

Constraint evaluation proceeds in priority order (fast-fail mode by default): (1) Sanctions constraints — unconditional PROHIBITED; (2) Jurisdiction-Asset constraints; (3) Jurisdiction-Operation constraints; (4) Ethical compliance constraints; (5) Client-specific constraints. Full-audit mode collects all violations before returning — configurable per deployment.

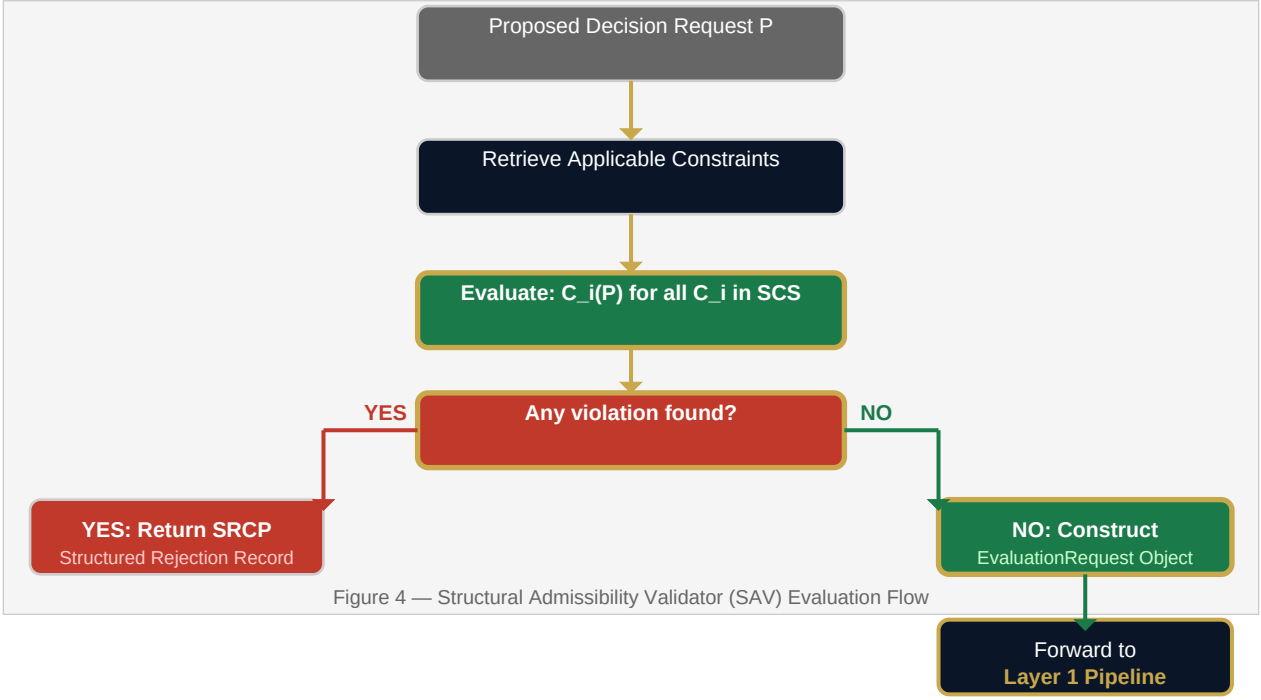

Figure 4 — Structural Admissibility Validator (SAV) Evaluation Flow

IV. ZERO-BYPASS BOUNDARY ENFORCEMENT (ZBE)

The zero-bypass property is the defining architectural characteristic distinguishing the SAE from all prior art runtime interception systems. It is enforced through private constructor restriction:

```
class EvaluationRequest: """Only constructible via
StructuralAdmissibilityValidator.validate_and_construct()""" _SAV_CONSTRUCTION_TOKEN =
object() # Private sentinel — never exposed externally def __init__(self, _token, asset,
operation, jurisdiction, client_id, metadata): if _token is not
EvaluationRequest._SAV_CONSTRUCTION_TOKEN: raise StructuralAdmissibilityViolation(
"EvaluationRequest must be constructed via "
"StructuralAdmissibilityValidator.validate_and_construct()" ) self.asset = asset
self.operation = operation self.jurisdiction = jurisdiction self.client_id = client_id
self.metadata = metadata class StructuralAdmissibilityValidator: def
validate_and_construct(self, proposed_request: dict) -> EvaluationRequest: violations =
self._evaluate_all_constraints(proposed_request) if violations: raise
StructuralAdmissibilityViolation( constraint_provenance=violations ) return EvaluationRequest(
_token=EvaluationRequest._SAV_CONSTRUCTION_TOKEN, **proposed_request )
```

Architectural guarantee: (a) No EvaluationRequest object can exist that has not passed through full SAV constraint evaluation. (b) No code path, configuration flag, or operational condition can produce an EvaluationRequest bypassing the SAV. (c) Any direct construction attempt raises an immediate exception that halts construction. **Silence is impossible:** an inadmissible request either raises an exception or is never constructed.

V. STRUCTURED REJECTION WITH CONSTRAINT PROVENANCE (SRCP)

When the SAV determines a proposed request is structurally inadmissible, it returns a machine-readable Structured Rejection Record with full constraint provenance:

```
{                                   "admissibility":
"INADMISSIBLE",                     "rejected_at":
"LAYER_0_STRUCTURAL_ADMISSIBILITY", "violations": [{
    "constraint_class":           "JURISDICTION_ASSET",
    "constraint_id":              "JA-UAE-XMR-001",
    "regulatory_source":          "UAE VARA Regulations 2023, Schedule 1",
    "input_fields":               ["asset", "jurisdiction"],
    "resolution":                 "Select VARA-compliant asset for UAE."
  }],                               "pipeline_entry":
false,                              "layer_0_time_ms":
0.8                                 }
```

Figure 5 — Structured Rejection Record example: XMR (Monero) in UAE jurisdiction, rejected at Layer 0 in 0.8ms. Machine-readable provenance identifies regulatory source, responsible fields, and resolution guidance.

The SRCP serves three purposes: (i) **Regulatory Audit** — complete machine-readable trail of why the decision was rejected, which regulatory source mandated the constraint, and which input fields produced the violation; (ii) **Client Communication** — the resolution field provides actionable guidance on reformulating the request; (iii) **System Debugging** — constraint_id and input_fields enable precise identification of constraint configuration errors.

VI. COMPOSABLE CROSS-DOMAIN CONSTRAINT ARCHITECTURE (CCCA)

All constraints are registered in a unified Constraint Registry at system initialization, indexed by the fields each constraint evaluates. For a proposed request P, the SAV retrieves all applicable constraints and

evaluates them as a conjunction — ADMISSIBLE iff no constraint returns PROHIBITED. New constraint classes are added to the registry without modifying the SAV evaluation logic, making the architecture extensible by constraint addition, not by code modification.

VII. MULTI-DOMAIN APPLICABILITY

The SAE is domain-agnostic. The constraint classes are illustrated using financial trading but apply without modification to all OMNIX governance domains:

Domain	SAE Constraint Classes Applied
Financial Trading	Asset + Operation + Jurisdiction + Sanctions + Client-Specific
Insurance Underwriting	Coverage line + Type + Jurisdiction + FHEO anti-discrimination
Medical AI	Diagnostic category + Treatment type + FDA/CE/MHRA approval status
Real Estate	Property type + Transaction type + AML structural constraints
Energy Governance	Energy source + Contract type + Grid operator jurisdiction + Carbon
Autonomous Agents	Agent action type + Domain + Authorization scope + Safety envelope

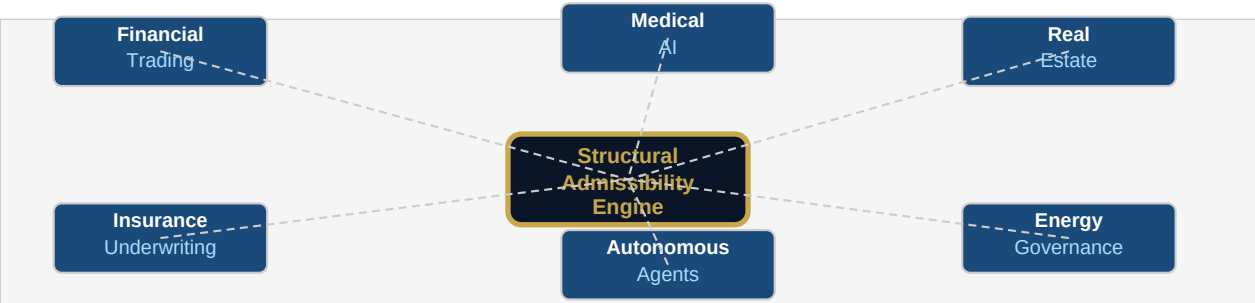

Figure 5 — SAE Multi-Domain Applicability (Domain-Agnostic Layer 0)

VIII. PERFORMANCE CHARACTERISTICS

Metric	Value	Notes
Layer 0 processing time (fast-fail)	0.4 – 2.1 ms	In-memory constraint tables
Layer 0 processing time (full-audit)	1.2 – 4.8 ms	All violations collected
Constraint table refresh	Asynchronous	Configurable interval; atomic update
Evaluation complexity	O(k)	k = applicable constraints for P's fields
Database query at evaluation time	None	In-memory; no DB query per request
Dynamic constraint refresh lag	≤30 seconds	Sanctions list updates; configurable

CLAIMS

1. A computer-implemented Structural Admissibility Engine for automated decision governance systems, comprising: (a) a Structural Constraint Schema encoding admissibility constraints organized into at least two constraint classes including jurisdiction-asset constraints and jurisdiction-operation constraints; (b) a Structural Admissibility Validator configured to evaluate a proposed decision request against the Structural Constraint Schema before constructing a valid EvaluationRequest object representing the proposed decision request; (c) wherein the Structural Admissibility Validator constructs the EvaluationRequest object only if the proposed decision request satisfies all applicable constraints in the Structural Constraint Schema; and (d) wherein the Structural Admissibility Validator does not construct the EvaluationRequest object if the proposed decision request violates any applicable constraint in the Structural Constraint Schema.

2. The Structural Admissibility Engine of claim 1, wherein the EvaluationRequest object type comprises a private constructor accessible exclusively through the Structural Admissibility Validator, enforcing a zero-bypass property whereby no code path external to the Structural Admissibility Validator can construct a valid EvaluationRequest object.

3. The Structural Admissibility Engine of claim 1, wherein the Structural Constraint Schema further comprises ethical compliance constraints derived from at least one of: Sharia compliance criteria, environmental, social, and governance (ESG) screening criteria, and international sanctions lists maintained by OFAC, the European Union, or the United Nations.

4. The Structural Admissibility Engine of claim 1, wherein the Structural Constraint Schema further comprises client-specific constraints encoding per-client admissibility restrictions, and wherein client-specific constraints may further restrict but may not expand the admissibility determined by jurisdiction-asset constraints and jurisdiction-operation constraints.

5. The Structural Admissibility Engine of claim 1, wherein when the Structural Admissibility Validator determines that a proposed decision request is inadmissible, the Validator returns a Structured Rejection Record comprising: a constraint identifier identifying the violated constraint; a constraint class identifier; a regulatory source citation identifying the regulatory or policy source from which the constraint derives; an identification of the specific input fields jointly responsible for the violation; and a resolution guidance field providing actionable reformulation instructions.

6. The Structural Admissibility Engine of claim 1, wherein the Structural Admissibility Validator supports a composable cross-domain constraint architecture in which constraints from multiple constraint classes are evaluated simultaneously through a unified Constraint Registry, and wherein new constraint classes are added to the registry without modification to the Structural Admissibility Validator evaluation logic.

7. The Structural Admissibility Engine of claim 1, wherein the Structural Admissibility Engine constitutes Layer 0 of a four-layer governance architecture, and wherein Layer 0 gates a Layer 1 runtime checkpoint pipeline that accepts only EvaluationRequest objects as input, ensuring that the Layer 1 pipeline cannot receive structurally inadmissible inputs regardless of the operational state of any Layer 1 checkpoint.

8. The Structural Admissibility Engine of claim 1, wherein constraint evaluation operates on in-memory constraint tables populated at system initialization, and wherein constraint table refresh for dynamic constraint classes including sanctions lists is performed asynchronously on a configurable refresh interval with atomic in-memory table updates, such that no database query is required at request evaluation time.

9. The Structural Admissibility Engine of claim 7, wherein the four-layer governance architecture further comprises Layer 2, a trajectory invariant enforcement layer evaluating the proposed decision against the historical and projected behavioral trajectory of the governance system; and Layer 3, a post-evaluation evidence and cryptographic receipt layer generating a post-quantum cryptographically sealed audit record using the Dilithium-3 (ML-DSA-65, NIST FIPS 204) signature scheme.

10. A method for enforcing structural admissibility in automated decision governance systems comprising: (a) receiving a proposed decision request comprising at least an asset identifier, an operation type, and a jurisdictional context; (b) evaluating the proposed decision request against a Structural Constraint Schema encoding at least jurisdiction-asset constraints and jurisdiction-operation constraints, the evaluation occurring prior to constructing any system object representing the proposed decision request; (c) if the proposed decision request satisfies all applicable constraints, constructing an EvaluationRequest object via a Structural Admissibility Validator and forwarding the EvaluationRequest object to a downstream governance pipeline; (d) if the proposed decision request violates any applicable constraint, returning a Structured Rejection Record identifying the violated constraints with constraint provenance and not constructing any EvaluationRequest object; and (e) enforcing a zero-bypass property by restricting EvaluationRequest object construction to the Structural Admissibility Validator.

ABSTRACT

A Structural Admissibility Engine (SAE) for automated decision governance systems enforces decision admissibility at the point of input construction rather than at runtime evaluation, making structurally inadmissible decision requests unrepresentable as valid system objects. The SAE comprises a Structural Constraint Schema (SCS) encoding regulatory, jurisdictional, and ethical constraints in four composable classes; a Structural Admissibility Validator (SAV) that evaluates proposed decision requests against the SCS before constructing any EvaluationRequest object; and a Zero-Bypass Boundary Enforcement (ZBE) mechanism that restricts EvaluationRequest construction exclusively to the SAV via private constructor restriction. Inadmissible requests are rejected at the structural boundary with a Structured Rejection Record providing machine-readable constraint provenance identifying the specific violated constraint, its regulatory source, and the responsible input fields. The SAE constitutes Layer 0 of a four-layer governance architecture that gates a downstream runtime checkpoint pipeline (Layer 1), trajectory invariant enforcement layer (Layer 2), and post-quantum cryptographic receipt layer (Layer 3). The zero-bypass property guarantees that no inadmissible request can enter the governance pipeline regardless of the operational state of any downstream component — a categorical architectural improvement over runtime interception systems where governance guarantees are contingent on every intercepting component being operational and correctly implemented. The SAE is domain-agnostic and applies to financial trading, insurance underwriting, medical AI, real estate, energy governance, and autonomous agent systems.